

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

Madhu

Department of Computer Science and Engineering
Indian Institute of Technology Bhilai
`madhu@iitbhillai.ac.in`

September 29, 2026

Abstract

Efficient rainwater harvesting and optimal pond site selection are essential for sustainable rural water management in India. Traditional manual site selection suffers from terrain complexity, unaggregated meteorological datasets, and lack of automated hydrologic analysis. In this project, we present **JalDrishti**, an AI-assisted geospatial web platform designed for village administrators and hydrologists. JalDrishti enables users to select land boundaries on an interactive high-resolution satellite map, automatically delineates the contributing catchment area using a D8 flow accumulation model on Digital Elevation Models (DEM), queries historical rainfall trends from Open-Meteo APIs, estimates surface runoff using the USDA Soil Conservation Service Curve Number (SCS-CN) method, and recommends the optimal pond location, depth, surface area, and volumetric storage capacity. The system features a high-performance FastAPI asynchronous backend, a responsive Leaflet.js Web GIS frontend, an in-memory spatial cache, and a resilient 24/7 watchdog daemon architecture deployed across allocated cluster nodes. Quantitative evaluations demonstrate an API response latency of under 220 ms for interactive queries and accurate runoff estimation, providing a robust solution for rural water conservation.

Keywords: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs, SCS-CN Model

1 Introduction

Water scarcity during dry seasons poses severe socio-economic challenges to rural agrarian communities across India. Rainwater harvesting via village ponds provides a decentralized, low-cost solution for groundwater recharge and supplementary irrigation. However, identifying optimal locations for constructing village ponds requires evaluating complex topographic gradients, watershed boundaries, local land availability, and multi-year precipitation patterns. Manual site selection by local authorities often leads to sub-optimal placement in high-permeability or low-catchment zones, resulting in low water retention and poor financial utilization.

To address these challenges, we built **JalDrishti**, an automated, web-based geospatial decision support system. The platform allows users to draw or select custom village land polygons directly on a map. Within seconds, the system computes the Digital Elevation Model (DEM) gradients, delineates the upstream catchment boundary, computes annual precipitation statistics, estimates surface runoff volume, and overlays the optimal pond location along with recommended physical dimensions (depth, width, capacity) directly on the interactive satellite map.

1.1 Motivation

Manual site selection for rural ponds is inherently error-prone due to:

1. **Terrain Complexity:** Topographic elevation data is often buried in large raw raster files or complex GIS tools (e.g., QGIS, ArcGIS) inaccessible to village panchayats.
2. **Meteorological Data Fragmentation:** Historical daily rainfall records are scattered across satellite databases and meteorological APIs, making manual aggregation labor-intensive.
3. **Lack of Automated Hydrological Modeling:** Calculating runoff volume requires integrating land cover, soil permeability, and slope gradients, which requires specialized GIS engineering expertise.

An automated Web GIS platform democratizes site selection, allowing panchayat engineers and researchers to instantly evaluate candidate land parcels with precision.

1.2 Scope of the Project

The JalDrishti system provides:

- Interactive drawing of village land polygons on high-resolution satellite basemaps.
- Automatic topographic contour generation (1-meter elevation intervals).
- D8 flow-direction and flow-accumulation catchment area delineation for selected sites.
- Automated fetching and statistical aggregation of 10-year historical daily precipitation data.
- Surface runoff volume calculation using the SCS-CN hydrological model.
- Recommended pond location, excavated volume, surface area, and optimal depth.
- Automated PDF report generation and visual GeoJSON map overlays.

Out of Scope: The platform focuses on hydrological and geospatial site recommendation; it does not perform structural civil engineering design (e.g., concrete reinforcement, soil compaction testing, embankment spillway sizing).

2 Problem Statement and Requirements

The primary objective of this assignment is to develop an end-to-end, fast, and scalable web system that enables users to inspect terrain, select land areas, and receive actionable pond design recommendations overlaid on a map.

2.1 Non-Functional Requirements

- **Performance:** Full end-to-end analysis response time must remain below 300 ms for standard land polygon selections.
- **Availability & Resilience:** The system backend must execute behind a 24/7 watchdog process with automatic restart capabilities, remaining active even during SSH disconnects.
- **Scalability:** Support concurrent REST queries across multiple cluster nodes using lightweight asynchronous event loops (`uvicorn` + `asyncio`).
- **Usability:** Mobile-responsive, dark-mode Web GIS interface with dynamic color-coded elevation contours, chart visualizations, and one-click export.

Table 1: Functional requirements and system implementation mapping

Functional Requirement	Implemented Module / File
Satellite imagery display	<code>static/app.js</code> , <code>static/index.html</code>
Contour map visualization	<code>app/main.py</code> , <code>contours_1m.kml</code> , <code>static/app.js</code>
Available-land identification	<code>static/app.js</code> (Leaflet Geoman), <code>app/main.py</code>
Catchment area estimation	<code>app/catchment.py</code> (D8 Flow Accumulation)
Historical rainfall query	<code>app/rainfall.py</code> (Open-Meteo Async Client)
Runoff volume estimation	<code>app/runoff.py</code> (SCS-CN Method)
Pond depth / storage recommendation	<code>app/pond_design.py</code> (Terrain Optimization)
Combined overlay / results view	<code>static/index.html</code> , <code>static/app.js</code>

3 System Architecture and High-Level Design

The JalDrishti system follows a decoupled, three-tier architecture comprising a Web GIS Client, an Asynchronous FastAPI REST Server, and External Spatial Data Services.

3.1 Technology Stack

- **Backend Framework:** Python 3.12, FastAPI 0.115.0, Uvicorn 0.30.6.
- **Geospatial & Mathematical Libraries:** SciPy 1.13.1, Shapely 2.0.4, NumPy 1.26.4, LXML 5.2.2.
- **Frontend GIS Stack:** Vanilla JavaScript (ES6+), HTML5, CSS3, Leaflet.js 1.9.4, Leaflet-Geoman 2.14.2, Chart.js 4.4.1.
- **External APIs:** Open-Meteo Historical Weather API, Esri World Imagery Tile Server.
- **Deployment & Networking:** Systemd / Bash 24/7 Watchdog Daemon, Localhost.run SSH Tunnels, IIT Bhilai Cluster NAT Port Mapping.

4 Methodology

4.1 Terrain and Elevation Analysis

Elevation data is extracted from standard Digital Elevation Models (DEM) and pre-processed 1-meter elevation KML contour datasets (`contours_1m.kml`). Contours are parsed using fast XML processing (`lxml`) into Shapely LineString geometries. The terrain slope S_{slope} at any point (x, y) is derived from elevation gradients:

$$S_{\text{slope}} = \sqrt{\left(\frac{\partial z}{\partial x}\right)^2 + \left(\frac{\partial z}{\partial y}\right)^2} \quad (1)$$

4.2 Catchment Area Delineation

We implement the D8 (Deterministic 8-neighbor) flow-direction model. For each grid cell in the DEM, water flows to the single steepest downward neighbor. Flow accumulation $A(x, y)$ is

recursively computed:

$$A(u, v) = 1 + \sum_{(i, j) \in \text{Upstream}(u, v)} A(i, j) \quad (2)$$

The total catchment area contributing to the selected polygon is delineated by identifying all cells that drain through the lowest elevation point inside the user-selected land boundary.

4.3 Rainfall Data Integration & Runoff Modeling

Historical daily precipitation P is fetched asynchronously for the geographic coordinates of the site from the Open-Meteo Historical Weather API across a multi-year window (2014–2024). Surface runoff volume Q (in mm) is computed using the USDA Soil Conservation Service Curve Number (SCS-CN) method:

$$Q = \frac{(P - I_a)^2}{P - I_a + S} \quad \text{for } P > I_a \quad (3)$$

where $I_a = 0.2S$ represents initial abstraction (canopy interception, surface storage), and S is the maximum potential retention capacity derived from the hydrologic Soil Group Curve Number $CN = 75$:

$$S = \frac{25400}{CN} - 254 \quad (4)$$

Total expected annual runoff volume V_{runoff} (in cubic meters, m^3) across catchment area $A_{\text{catchment}}$ (in m^2) is:

$$V_{\text{runoff}} = \frac{Q_{\text{annual}}}{1000} \times A_{\text{catchment}} \quad (5)$$

4.4 Optimal Pond Site Selection and Sizing

The system searches within the selected available land polygon for the local topographic minimum (lowest elevation cell with favorable slope $< 5\%$) to minimize excavation earthwork costs. Recommended pond depth D_{pond} and surface area A_{pond} are optimized to capture 60%–80% of average annual runoff while maintaining a maximum depth of 3.5 m to prevent thermal stratification and collapse:

$$V_{\text{storage}} = A_{\text{pond}} \times D_{\text{pond}} \quad (6)$$

5 Implementation

5.1 Backend and REST API Design

The backend exposes clean REST endpoints using FastAPI, featuring built-in CORS headers and JSON schemas.

Table 2: Backend REST API Endpoints

HTTP Method	Path	Purpose / Description
POST	/analyzeSelectedArea	Computes catchment, rainfall, runoff, and pond design
GET	/contours	Returns 1m interval GeoJSON elevation contours
GET	/api/health	Liveness check endpoint returning system status
GET	/	Serves the main Web GIS dashboard single-page application

Table 3: Mapping of core Computer System Design (CSD) themes to system implementation

CSD Theme/Topic	Where used in project		Justification / Design Rationale
REST API Design	app/main.py		Standardized HTTP methods, JSON payload structures, CORS support.
Load Balancing & NAT	Cluster	Nodes	Distributed multi-port binding (3000,6000) mapping via cluster NAT (3313..3316, 6313..6316).
Caching	app/rainfall.py		In-memory LRU cache for external Open-Meteo rainfall queries to reduce latency.
Concurrency / Async	httpx, uvicorn		Non-blocking asynchronous event loop prevents I/O blocking during API calls.
Fault Tolerance & Resilience	start_persistent_server.sh		24/7 background watchdog loop auto-restarts failed server instances in < 2s.
Algorithms & Complexity	app/catchment.py		$O(N \log N)$ D8 flow accumulation algorithm for fast spatial grid processing.
Containerless Deployment	System Python / Tarball		Complies with cluster storage limits (< 1.5 GB) by avoiding heavy Docker images.
Testing Strategy	test_app_unit.py, test_api.py		Automated unit and endpoint integration testing suite.

5.2 Frontend and Visualization

The single-page web app (`static/index.html`, `static/app.js`) utilizes Leaflet.js layered over satellite basemaps. Users can draw custom land boundaries using Leaflet-Geoman controls. Upon submitting an area, the frontend sends the GeoJSON geometry to `/analyzeSelectedArea` and renders:

1. **Pond Location Marker:** Visualized with optimal dimensions and lat/long coordinates.
2. **Catchment Boundary Overlay:** Color-coded polygon representing the upstream watershed.
3. **Analytical Dashboard Panel:** Displays catchment area (km^2), annual rainfall (mm), estimated runoff (m^3), recommended depth (m), and pond capacity (m^3).
4. **Rainfall Chart:** Multi-year trend bar chart powered by Chart.js.

6 CSD Themes and Topics Applied in the Project

6.1 Engineering Rationale for Key CSD Decisions

1. **Resilient 24/7 Watchdog Daemon over Heavy Virtual Environments:** Given the cluster storage restriction (1.5 GB per system limit), traditional Docker containers or heavy Anaconda environments were unsuitable. We implemented a lightweight shell watchdog loop (`start_persistent_server.sh`) using system Python and `nohup/disown` daemons, ensuring 24/7 zero-downtime availability.
2. **Asynchronous Non-Blocking I/O:** Querying multi-year daily precipitation from external web APIs can introduce 1–2 second network overhead. By leveraging FastAPI's

native `async/await` event loop, the server handles hundreds of concurrent map queries without worker thread starvation.

7 Results and Evaluation

The system was evaluated using representative rural land parcels in Chhattisgarh, India.

Table 4: Sample Analysis Output for a 0.85 km² Test Parcel

Metric	Computed Value
Selected Land Area	0.850 km ² (85,000 m ²)
Delineated Catchment Area	1.420 km ²
Average Annual Rainfall	1,185.4 mm
Estimated Annual Runoff	364,500 m ³
Recommended Pond Location	Lat: 21.1842°N, Lon: 81.7135°E
Optimal Pond Depth	3.00 m
Recommended Surface Area	12,000 m ² (120 m × 100 m)
Target Storage Capacity	36,000 m ³
End-to-End API Response Time	215 ms

7.1 Performance Under Stress

Under benchmark testing with 50 concurrent requests using `pytest` and `httpx`, the FastAPI backend maintained an average response latency of 218 ms with 0% error rate.

8 Discussion and Limitations

JalDrishti provides rapid, scientifically grounded pond site recommendations. However, a few limitations exist:

1. **DEM Grid Resolution:** Standard open DEM datasets have a 30-meter grid resolution, which may smooth out minor micro-topographic bunds.
2. **Soil Permeability Assumption:** The system currently uses a default Curve Number ($CN = 75$) corresponding to typical agricultural loam soils; real-world deployment would benefit from site-specific soil testing inputs.

9 AI Tool Usage Declaration

In accordance with the assignment’s LLM Usage Policy, we declare that generative AI tools (Antigravity coding assistant, Claude Opus 4.6, Gemini 3.6 Flash) were utilized for:

- Reformatting and organizing the LaTeX report into the required manuscript structure.
- Brainstorming UI/UX layout optimizations for the Leaflet Web GIS frontend.
- Refactoring asynchronous API handler functions in FastAPI.

All AI-generated code snippets and report contents were thoroughly reviewed, verified, tested, and integrated by the author.

A Source Code, Repository, and Allocated System URLs

A.1 Source Code Repository

- **GitHub Repository URL:** <https://github.com/madhu328/pond-catchment-api>

A.2 IIT Bhilai Cluster Port Mapping & System Access Links

According to the cluster networking rules, an internal app port $P \in \{2000, 3000, 4000, 5000, 6000, 7000\}$ configured on an allocated system (IP: 10.1.75.51, SSH Port: 2000 + System_ID) is mapped across the IIT Bhilai intranet at http://10.1.75.51:<P+System_ID>.

Port 3000 is allocated for the **Backend REST API** (Port 3313), while Port 6000 is allocated for the **Web GIS Frontend Application** (Port 6313).

Table 5: System Access and Port Mapping Matrix (Backend API vs Frontend Web UI)

Component	Internal Port	Mapped Intranet URL	Purpose
Backend Primary API	Port 3000	http://10.1.75.51:3313/analyzeContour	Primary I
Backend Alias API	Port 3000	http://10.1.75.51:3313/findCatchment	Alias Cat
Backend Map API	Port 3000	http://10.1.75.51:3313/analyzeSelectedArea	Interactiv
Backend Swagger UI	Port 3000	http://10.1.75.51:3313/docs	OpenAPI
Frontend Web App	Port 6000	http://10.1.75.51:6313/	Interactiv

A.3 Working URLs Summary

- **Working Backend API URL:** <http://10.1.75.51:3313/analyzeContour>
- **Working Frontend Web App URL:** <http://10.1.75.51:6313/> (also available locally at <http://localhost:6313/>)